

4 DESIGN

Date

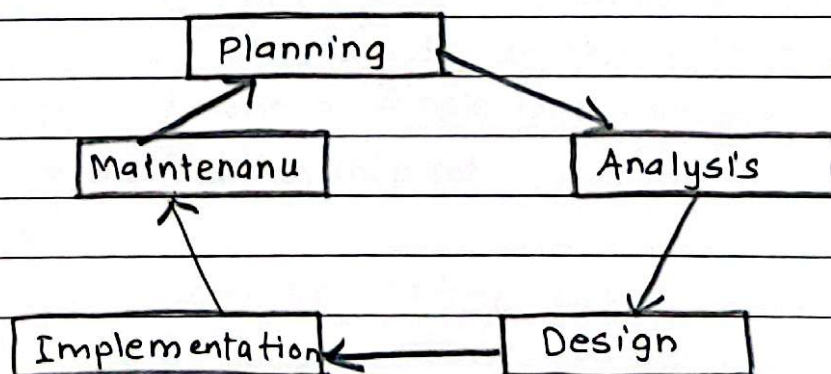
System designing is a process of planning a new business system or replacing an existing system by defining its components or modules to satisfy the specific requirements.

System design is the phase that bridges the gap between problem domain and the existing system in a manageable way. This phase focus on the solution domain, i.e. "how to implement?". It is the phase where the SRS document is converted into a format that can be implemented and decides how the system will operate.

Database Design

Database design is the organization of data according to a database model. The designer determines what data must be stored and how the data elements interrelate. Database design is a collection of processes that facilitate the designing, development, implementation and maintenance of enterprise data management systems. It helps produce database systems that meets the requirements of the user while having high performance.

The main objectives of database designing are to produce logical and physical designs models of the proposed database system.



• Databases
Forms & reports
Dialogues
Interviews

fig: SDLC with design phase highlighted

The Process of Database Design

- Enterprise wide data model (ER with only attributes)
- Conceptual data model (ER with only attribute for specific project)

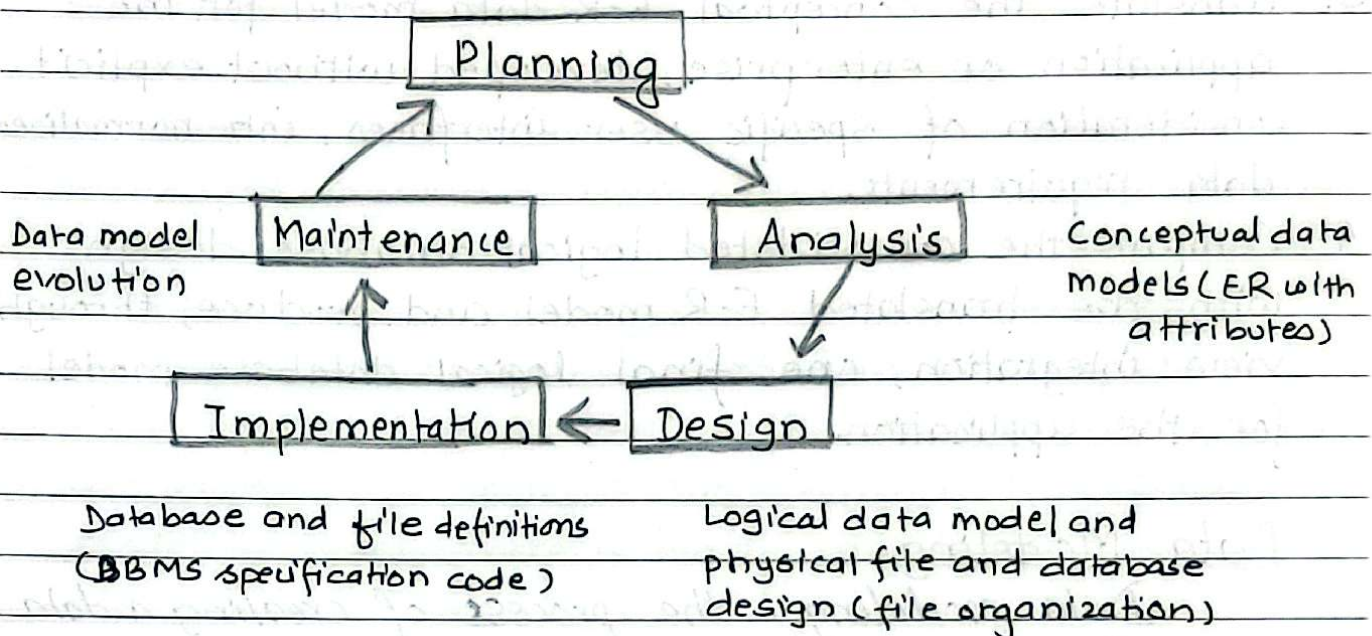


fig: Relationship between data modeling and the SDLC.

Database modeling and design activities occur in all phases of systems development process. In design phase, the logical and physical databases are designed. In logical database design, we use a process called normalization, which is a way to build a data model that has the properties of simplicity, non redundancy and minimal maintenance.

In most situations, many physical database design decisions are implicit or eliminated when we choose the data management technologies to use with the application.

There are four key steps in logical database modeling and design:

- 1) Develop a logical data model for each known user interface (form and report) for the application using

normalization principles.

2. Combine normalized data requirements from all user interfaces into one consolidated logical database model; this step is called view integration.
3. Translate the conceptual E-R data model for the application or enterprise, developed without explicit consideration of specific user interfaces, into normalized data requirements.
4. Compare the consolidated logical database design with the translated E-R model and produce, through view integration, one final logical database model for the application.

Data Modeling

Data modeling is the process of creating a data model for the data to be stored in a database. This data model is a conceptual representation of

- data objects
- The associations between different data objects
- The rules.

Uses of Data Model

- Ensures that all data objects required by the database are accurately represented.
- A data model helps design the database at the conceptual, logical and physical levels.
- Data model structure helps to define the relational tables, primary and foreign keys and stored procedures.
- It provides a clear picture of the base data and can be used by database developers to create a physical database.
- It is helpful to identify missing and redundant data.
- It makes upgrade and maintenance cheaper and faster.

Types of Data Models :

There are mainly three different types of data models :

- conceptual model → logical data model
- Physical data model.

i) Conceptual data model : This data model defines what the system contains. This model is typically created by business stakeholders and Data Architects. The purpose is to organize scope and define business concepts and rules. The main aim of this model is to establish the entities, their attributes and their relationships. In this Data modeling level, there is hardly any detail available of the actual database structure.

Characteristics :

- Offers organization-wide coverage of the business concepts.
- Designed and developed for a business audience.
- The conceptual model is developed independently of hardware specifications and software specifications. The focus is to represent data as user will see it in the "real world".

ii) Logical data model : It defines how the system should be implemented regardless of the DBMS. This model is typically created by Data Architects and Business Analysts. The purpose is to develop technical map of rules and data structures. Logical data models add further information to the conceptual model elements.

Characteristics:

- Describes data needs for a single project but could integrate with other logical data models based on the scope of project.
- Designed and developed independently from the DBMS.
- Data attributes will have data types with exact precisions and length.
- Normalization processes to the model is applied typically till 3NF.

iii) Physical data model: This data model describes how the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.

Characteristics:

- Describes data need for a single project or application, it may be integrated with other physical data models based on project scope.
- Contains relationships between tables that which addresses cardinality and null ability of the relationships.
- Developed for a specific version of a DBMS, location, data storage or technology to be used in the project.
- Columns should have exact datatypes, lengths assigned and default values.
- Primary and Foreign keys, views, indexes, access profiles, and authorizations, etc. are defined.

Advantages of Data Model:

- The main goal of designing a data model is to make certain that data objects offered by the functional team are represented accurately.
- The data model should be detailed enough to be used for building the physical database.
- The information in the data model can be used for defining the relationship between tables, primary and foreign keys, and stored procedures.
- Data Model helps business to communicate within and across organizations.
- Data model helps to documents data mappings in ETL (Extract, Transform and Load) process.
- Help to recognize correct sources of data to populate the model.

Disadvantages of Data Model:

- To develop Data Model one should know physical data stored characteristics.
- Even smaller changes made in structure require modification in the entire application.
- There is no set data manipulation language in DBMS.

Relational Database Model:

The relational database model represents data in the form of related tables, or relations.

A relation is a named, two-dimensional table of data. Each relation (or table) consists of a set of named columns and an arbitrary number of unnamed rows. Each column in a relation corresponds to an attribute of that relation. Each row of a relation corresponds to a record that contains data values for an entity.

Example: ^{Primary key attribute} STUDENT ^{Columns (Attributes)}

Records (tuples)	Stu_id	Stu_name	Stu_address	Dept_id
1	12	Ramesh	Butwal	5
1	13	Shyam	Kathmandu	3
1	14	Hari	Bhalrahawa	1
1	15	Sita	Pokhara	4

Characteristics that distinguish Relations from nonrelational tables:

1. Entries in cells are simple. An entry at the intersection of each row and column has a single value.
2. Entries in a given column are from the same set of values.
3. Each row is unique. Uniqueness is guaranteed because the relation has a non-empty primary key value.
4. The sequence of columns can be interchanged without changing the meaning or use of the relation.
5. The rows may be interchanged or stored in any sequences.

Some terms used in relational database:

- * Attribute: Definite properties of the relation.
- * Tuple: Row of a relation that corresponds to a record.
- * Domain: Set of values which is indivisible i.e. value for each attribute present in the table contains some specific domain in which the value needs to lie.

Normalization

Normalization is the process of converting (restructuring) complex data structures into simple, stable data structures in accordance with a series of so-called normal forms in order to reduce data redundancy and improve data integrity. Redundancy in relation may cause insertion, deletion and updation anomalies. Normalization can be considered as a "filtering" or "purification" process to make the design have successively better quality.

Normal Forms: The normal form of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

De-Normalization: It is the process of adding redundant data to speed up complex queries involving multiple table JOINS.

Functional Dependency: A function dependency is a particular relationship between two attributes. In a given relation, attribute B is functionally dependent on attribute A, if for every valid value of A, that

value of A uniquely determines the value of B .

i.e. A function dependency denoted by $A \rightarrow B$, between two sets of attributes A and B that are subsets of R , such that any two tuples t_1 and t_2 that have $t_1[A] = t_2[A]$, they must also have $t_1[B] = t_2[B]$. Functional dependency is represented as FD or f.d.

1) First Normal Form (1NF): For a relation to be in the first normal form, it should follow the following rules:-

a) It should only have single (atomic) valued attributes/columns.

b) All the columns in a table should have unique names.

Consider the following table

Student

Sid	Sname	Salutation	Address	Phone	Sub_id	Sub_name
1	Ram	Mr	BHW	9847010382 9857030182	1	DB
2	Shyam	Mr	KTM	9824912345	1	DB
3	Gita	Mrs	BHW	9812912345	2	Java
4	Sita	Ms	KTM	9825612723	3	C-prog
2	Shyam	Mr	KTM	9824912345	2	Java

fig:- Table-1

Above table doesnot satisfy 1NF, because Column phone contains multiple values. Hence we need to create new table contact to store phone numbers.

Following are the normalized tables that satisfy 1NF.

Student					
Sid	Sname	Salutation	Address	Sub-id	Sub-name
1	Ram	Mr	BHW	1	DB
2	Shyam	Mr	KTM	1	DB
3	Gita	Mrs	BHW	2	Java
4	Sita	Ms	KTM	3	C-prog
2	Shyam	Mr	KTM	2	Java

Table - 2

Contact		
Contact-id	Phone	Sid
1	9847010382	1
2	9857030182	1
3	9824912345	2
4	9812912345	3
5	9825612723	4

Table - 3

Second Normal Form (2NF)

For a table to be in the Second Normal form, it must satisfy following conditions:

- The table should be in the First Normal form.
- There should be no partial dependency.

Note: * Partial function Dependency occurs only in relation with composite keys.

* It occurs when one or more non key attributes are depending on a part of the primary key.

Our Student table (Table-2) does not satisfy 2NF, because we have
primary key = Sid + Sub-id

Here, to determine name of subject, we use Sub-id which is part of primary key. Hence, Sub-name is partially dependent on Sub-id.

So we need to remove partial dependency from student table and create new table Subject to store subject details.

Normalized tables:

Student

Sid	Sname	Salutation	Address	
1	Ram	Mr	BHW	also,
2	Shyam	Mr	KTM	Sid → Sname →
3	Gita	Mrs	BHW	Salutation → Address
4	Sita	Ms	KTM	(partial dependent on
2	Shyam	Mr	KTM	Sid)

Table - 4

Subject

Sub-id	Sub-name
1	DB
2	Java
3	C-prog

Table - 5a

Enrollment

Sid	Sub-id	
1	1	
2	1	
3	2	Table-5b
4	3	
2	2	

Third Normal Form (3NF):

For a table to be in Third Normal Form it must satisfy following requirements:

- A table is already in 2NF.
- There should be no transitive dependencies.

*note: Transitive dependency:- Non-primary key attributes depending on other non-primary key attributes.

In our table 4, there is transitive dependency on Sname and Salutation. Both are non-primary key attributes. Change in Sname might cause change in Salutation.

Normalized tables:-

Student

Sid	Sname	Sal-id	Address
1	Ram	1	BHW
2	Shyam	1	KTM
3	Gita	3	BHW
4	Sita	2	KTM
2	Shyam	1	KTM

Table - 6

Salutation

Sal-id	Salutation
1	Mr.
2	Ms
3	Mrs

Table - 7

Transforming E-R Diagrams into Relations:

Transforming an E-R diagram into normalized relations and then merging all the relations into one final, consolidated set of relations can be accomplished in four steps:

- 1) Represent entities: Each entity type in the E-R diagram becomes a relation. The identifier of the entity type becomes the primary key of the relation, and other attributes of the entity type become non primary key attributes of the relation.

2) Represent relationships: Each relationship in an E-R diagram must be represented in the relational database design. How we represent a relationship depends on its nature. For example, in some cases we represent a relationship by making the primary key of one relation a foreign key of another relation. In other cases, we create a separate relation to represent a relationship.

3) Normalize the relations: The relations created in steps 1 and 2 may have unnecessary redundancy. So we need to normalize these relations to make them well structured.

4) Merge the relations: Across different sets of relations created in above steps, there may be redundant relations (two or more relations that describe the same entity type) that must be merged and renormalized to remove the redundancy.

Transforming different types of entities and relationships (E-R Structures):

a) Regular entity: Create a relation with primary key and nonkey attributes.

b) Weak entity: Create a relation with a composite primary key (which includes the primary key of the entity on which this weak entity depends) and nonkey attributes.

- c) Binary or unary 1:1 relationship: Place the primary key of either entity in the relation for the other entity or do this for both entities.
- d) Binary 1:N relationship: Place the primary key of the entity on the one side of the relationship as a foreign key in the relation for the entity on the many side.
- e) Binary or unary M:N relationship: Create a relation with a composite primary key using the primary keys of the related entities, plus any nonkey attributes associative entity of the relationship or associative entity.
- f) Binary or unary M:N relationship or associative entity with additional key(s): Create a relation with a composite primary key using the primary keys of the related entities and additional primary key attributes associated with the relationship or associative entity, plus any nonkey attributes of the relationship or associative entity.
- g) Binary or unary M:N relationship or associative entity with its own key: Create a relation with the primary key associated with the relationship or associative entity, plus any nonkey attributes of the relationship or associative entity and the primary keys of the related entities (as foreign key attributes.)

h) Supertype/subtype: Create a relation for the superclass, which contains the primary relationship key and all nonkey attributes in common with all subclasses, plus create a separate relation for each subclass with the same primary key (with the same or local name) but with only the nonkey attributes related to that subclass.

Merging Relations

Some of the relations may be redundant, we should merge those relations to remove the redundancy.

An example of Merging Relations:

Suppose modeling a user interface or transforming an E-R diagram results in following 3NF relation:

Employee 1 (Emp-ID, Name, Address, Phone)

Modeling a second user interface might result in following relation:

Employee 2 (Emp-ID, Name, Address, Jobcode, Number-of-years)

Because these two relations have same primary key (Emp-ID) and describe the same entity, they should be merged into one relation. The result of merging the relations is the following relation:

Employee (Emp-ID, Name, Address, Phone, Jobcode, Number-of-years).

Physical file and Database design:

Designing physical files and databases requires certain information that should have been collected and produced during prior SDLC phases. This information includes:

- Normalized relations, including volume estimates.
- Definitions of each attribute.
- Descriptions of where and when data are used: entered, retrieved, deleted, and updated (including frequencies)
- Expectations or requirements for response time and data integrity.
- Descriptions of the technologies used for implementing the files and database so that the range of required decisions and choices for each is known.

We take a bottom-up approach to reviewing physical file and database design. Thus we begin the physical design phase by addressing the design of physical fields for each attribute in a logical data model.

Designing Fields: A field is the smallest unit of application data recognized by system software, such as a programming language or database management system. An attribute from a logical database model may be represented by several fields. The basic decisions we must make in specifying each field concern the type of data (or storage type) used to represent the field and data integrity controls for field. Such as.

Choosing Data types: Selecting a data type balances four objectives that will vary in degree of importance, depending on the application:

1. Minimize Storage space.
2. Represent all possible values of the field.
3. Improve data integrity for the field.
4. Support all data manipulations desired on the field.

Calculated Fields: A field that can be derived from other database fields is called a calculated field. Some database technologies allow us to explicitly define calculated fields among with other raw data fields. The database technology will either store the calculated value or compute it when required.

Coding and Compression Techniques: We can reduce storage space ~~be~~ and increase integrity (by restricting input to only a few values), which helps to achieve two of the physical file and database design goals by using codes for some attributes that have very few values from a large range of possible values.

Controlling Data Integrity: Four popular data integrity control methods:

- a) Default value
- b) Range control
- c) Referential integrity
- d) Null value control

Designing Physical Tables: A physical table is a named set of rows and columns that specifies fields in each row of the table. The design of a physical table has two goals: efficient use of secondary storage and data processing speed.

→ The efficient use of secondary storage relates to how data are loaded on disks. The efficiency of secondary storage depends on factors such as operating system parameters, outside the control of each database.

→ Data are most efficiently processed when they are stored close to one another in secondary memory, thus, minimizing the number of input/output operations that must be performed.

Designing Forms and Reports:

Form: A business document that contains some predefined data and may include some areas where additional data are to be filled in. An instance of a form is typically based on one database record.

Report: A business document that contains only predefined data; it is a passive document used solely for reading or viewing. A report typically contains data from many unrelated records or transactions.

The forms are used to collect or present information on a single item, such as customer, product or event. Forms can be used for both inputs and output. Reports on the other hand are used to convey information on a collection of items. Form and report design is a key ingredient for successful systems. Because users often equate the quality of a system with the quality of its input and output methods, so design process for forms and reports is an especially important activity. And because information can be collected and formatted in many ways, gaining an understanding of design do's and don'ts and the trade-offs between various formatting options is very useful for all systems analysts.

Process of Designing Forms and Reports:

Designing forms and reports is a user-focused activity that typically follows prototyping approach. First we must gain an understanding of the intended user and task objectives by collecting initial requirements. During this process, several questions to "who, what, when, where and how" must be answered:

1. Who will use the form or report?
2. What is the purpose of the form or report?
3. When is the form or report needed and used?
4. Where does the form or report need to be delivered and used?
5. How many people need to use or view the form or report?

Answer to these questions will provide guidance for both the format and content of the design.

Gaining and understanding of these questions is a required first step in creation of any form or report. After collecting initial requirements, we structure and refine this information into an initial prototype. Now, we ask users to review and evaluate the prototype. After reviewing the prototype, users may accept the design or request some changes. If changes are needed we will refine it according to user requirements until the design is accepted.

Deliverables and Outcomes:

Design specifications are the major deliverables and are inputs to the system implementation phase of the designing forms and reports activity. Design specifications have three sections:

- i) Narrative overview: It contains overview of characteristics of target users, tasks, system etc in which form or report will be used.
- ii) Sample design: It provides sample design which may be hand drawn using a coding sheet although, in most instances, it is developed using CASE tool.
- iii) Testing and Usability Assessment: This section provides all testing and usability assessment information. Assessing usability depends on speed, accuracy and satisfaction.

Formatting Forms and Reports:

1) General Formatting Guidelines: Over the past several years, industry and academic researchers have investigated how the format of information influences individual task performance and perceptions of usability. Through this work, several guidelines for formatting information have emerged. They are:

i) Meaningful Titles:

- Clear and specific titles describing content and use of form and report
- Revision date or code to distinguish a form or report from prior versions
- Current date, which identifies when the form or report was generated.
- Valid date, which identifies on what date (or time) the data in the form or report were accurate

ii) Meaningful Information:

- Only needed information should be displayed
- Information should be provided in a manner that is usable without modification.

iii) Balance the Layout:

- Information should be balanced on the screen or page.
- Adequate spacing and margins should be used.
- All data and entry fields should be clearly labeled.

iv) Design an Easy Navigation System:

- clearly show how to move forward and backward
- Clearly show where you are [e.g. page 1 of 3]
- Notify user when on the last page of a multipaged sequence.

2) Guidelines for displaying Contents: The way the form or a report appears to the human eye has a lot of impact on the user so, we follow some guidelines for better display of contents:

i) Highlighting Information: There are several situations when highlighting can be a valuable technique for conveying special information:

- Notifying users of errors in data entry or processing.
- Providing warnings to user regarding possible problems such as unusual data values or an unavailable device.
- Drawing attention to keywords, commands, high-priority messages, and data that have changed or gone outside normal operating ranges.

Methods of highlighting

- Blinking and audible tones. → Boxing
- Color differences → Reverse video
- Intensity differences → Underlining
- Size differences → All capital letters
- font differences

ii) Using colour: Colour is a powerful tool for the designer in influencing the usability of a system. When applied appropriately, colour provides many potential benefits to forms and reports.

Benefits from using Colour:

- Soothes or strikes the eye.
- Accents on uninteresting display.
- Facilitates subtle discriminations in complex displays.
- Emphasizes the logical organization of information.
- Draws attention to warnings.
- Evokes more emotional reactions.

iii) Displaying text:

Guidelines for displaying text:

- * Case: Display text in mixed uppercase and lowercase and use conventional punctuation.
- * Spacing: Use double spacing if space permits. If not, place a blank line between paragraphs.
- * Justification: Left justify text and leave a ragged-right margin.
- * Hyphenation: Do not hyphenate words between lines.
- * Abbreviations: Use abbreviations and acronyms only when they are widely understood by users and are significantly shorter than the full text.

iv) Displaying Tables and Lists:

Guidelines for Displaying Tables and Lists:

Use Meaningful Labels:

- All columns and rows should have meaningful labels.
- Highlight labels.
- Redisplay labels, if data extend beyond a single screen or page.

Formatting Columns, Rows and Text:

- Sort in a meaningful order (e.g. ascending, descending etc.)
- Place a blank line between every five rows in long columns.
- Columns should have at least two spaces between them.
- Allow white space on printed reports for user to write notes.
- Use a single typeface, except for emphasis.
- Avoid overly fancy fonts.

Formatting Numeric, Textual, and Alphanumeric Data:

- Right justify numeric data and align columns by decimal points or other delimiter.
- Left justify textual data. Use short line length, usually 30-40 characters per line.
- Break long sequences of alphanumeric data in small groups of three to four characters each.

* Use tables for reading individual data values in reports.

* Use Graphs for:

- providing a quick summary of data.
- Detecting trends over time.
- Forecasting activities
- Comparing points and patterns of different variables
- Reporting vast amounts of information when relatively simple impressions are to be drawn.

Paper Vs Electronic Reports:

- | paper based reports | Electronic Reports |
|---|--------------------|
| 1) Paper based reports are stored locally in filing cabinets. | 1) |

Paper Versus Electronic Reports:

When a report is produced on paper rather than on a computer display, there are some additional things that we need to consider. For example, if paper reports are printed using impact printers, we use a coding sheet for designing forms and reports. We should follow prototyping process and carefully control the spacing of characters in order to produce a high quality report.

Assessing Usability:

Usability is an overall evaluation of how a system performs in supporting a particular user for a particular task.

- It typically refers to the following 3 characteristics
1. Speed: Can you complete a task efficiently?
 2. Accuracy: Does the output provide what you expect?
 3. Satisfaction: Do you like using the output?

General design guidelines for Usability of forms and Reports.

Usability factor Guidelines for Achievement of Usability.

Consistency → Consistent use of terminology, abbreviations, formatting, titles and navigation within and across outputs. Consistent response time each time a function is performed.

Efficiency → Formatting should be designed with an understanding of the task being performed and the intended user. Text and data should be aligned and sorted for efficient navigation and entry. Entry of data should be avoided where possible.

Ease → Outputs should be self-explanatory and not require users to remember information from prior outputs in order to complete tasks. Labels should be extensively used, and all scales and units of measure should be clearly indicated.

Format → Information format should be consistent between entry and display. Format should distinguish each piece of data and highlight, not bury, important data. Special symbols, such as decimal places, dollar signs, and $\pm$ signs, should be used as appropriate.

Flexibility → Information should be viewed and retrieved in a manner most convenient to the user. For example, users should be given options for the sequence in which to enter or view data and for use of shortcut keystrokes, and the system should remember where the user stopped during the last use of the system.

Designing Interfaces and Dialogues:

Interface design focuses on how information is provided to and captured from users. Dialogue design focuses on the sequencing of interface displays. The design of interfaces and dialogues is the process of defining the manner in which humans and computers exchange information.

Measures of Usability:

- i> Learnability: How difficult is it for a user to perform task for the first time?
- ii> Efficiency: How quickly can users perform tasks?
- iii> Error rate: How many errors might a user encounter, and how easy it is to recover from those errors?
- iv> Memorability: How easy is it to remember how to accomplish a task?
- v> Satisfaction: How enjoyable is the system to use?

Process of Designing Interfaces and dialogues:

Similar to designing forms and reports, the process of designing interfaces and dialogues is a user-focused activity. This means we should follow a prototyping methodology iteratively collecting information, and making refinements. This process parallels that of designing forms and reports.

Deliverables and Outcomes:

The deliverable and outcome from system interface and dialogue design is the creation of a design specification. Similar to that of forms and reports it contains three sections.

1. Narrative overview
2. Sample design
3. Testing and Usability assessment.

Design Specification

1. Narrative overview
 - a. Interface / Dialogue Name
 - b. User Characteristics
 - c. Task Characteristics
 - d. System Characteristics
 - e. Environmental characteristics
2. Interface / Dialogue Designs
 - a. Form / Report Designs
 - b. Dialogue Sequence Diagram(s) and Narrative Description.
3. Testing and Usability Assessment
 - a. Testing Objectives
 - b. Testing Procedures
 - c. Testing Results
 - i) Time to Learn
 - ii) Speed of performance
 - iii) Rate of Errors
 - iv) Retention over Time.
 - v) User Satisfaction and Other Perceptions.

fig:- Specification Outline for the design of interfaces and dialogues.

Interaction Methods And devices:

Methods of Interacting: There are numerous approaches for designing the interaction, some of most commonly used are:-

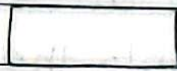
i) Command Language Interaction: In command language interaction, the user enters explicit statements to invoke operations within a system. This type of interaction requires users to remember command syntax and semantics.

CLI places a substantial burden on the user to remember names, syntax, and operations. Most newer or large-scale systems no longer rely entirely on a command language interface. Yet Command languages are good for experienced users, for systems with a limited command set, and for rapid interaction with the system.

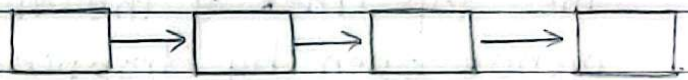
ii) Menu Interaction: It is a human-computer interaction ^{method} in which a list of system options is provided and a specific command is invoked by user selection of a menu option.

Menus have become the most widely used interface method because the user only needs to understand simple signposts and route options to effectively navigate through a system.

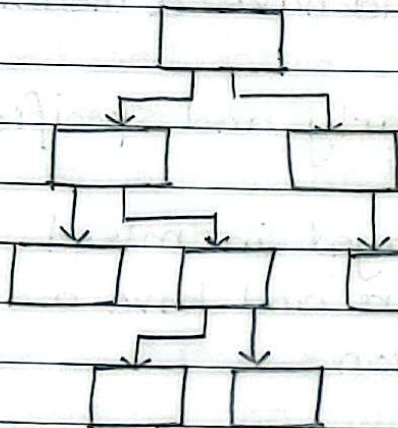
Various types of menu configurations :



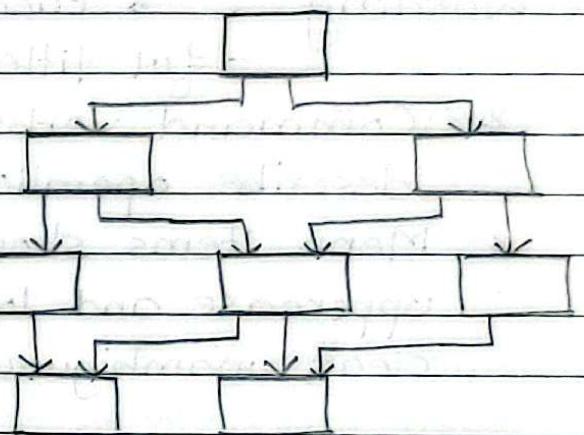
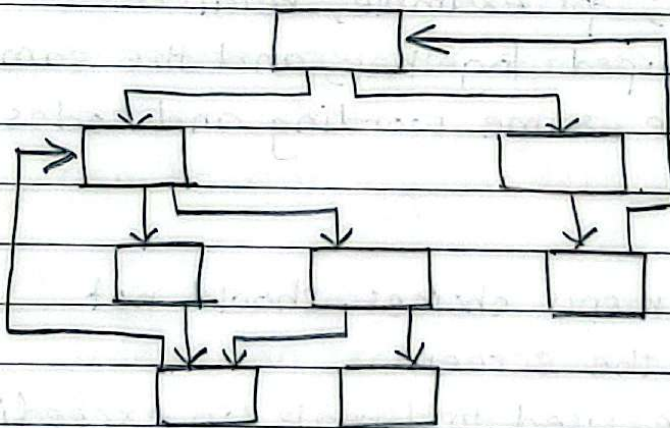
Single Menu



Linear Sequence Menu



Multilevel Tree Menu

Multilevel Tree Menu
With Multiple ParentsMultilevel Tree Menu with
Multiple Parents and
Multilevel Traversal

Methods for positioning menus: There are two common methods for positioning menus.

↳ Pop-up menu: A menu-positioning method that places a menu near the current cursor position.

ii) Drop-down menu: A menu-positioning method that places the access point of the menu near the top lines of the display; when accessed, menus open by dropping down onto the display.

Guidelines for Menu Design

Wording → Each menu should have a meaningful title.

→ Command verbs should clearly and specifically describe operations.

→ Menu items should be displayed in mixed uppercase and lowercase letters and have a clear, unambiguous interpretation.

Organization → A consistent organizing principle should be used that relates to the tasks the intended users perform; for example, related options should be grouped together, and the same option should have the same wording and codes each time it appears.

Length → The number of menu choices should not exceed the length of the screen.

→ Submenus should be used to break up exceedingly long menus.

Selection → Selection and entry methods should be consistent and reflect the size of the application and sophistication of the users.

→ How the user is to select each option and the consequences of each option should be clear (e.g., whether another menu will appear).

Highlighting -> Highlighting should be ~~not~~ minimized and used only to convey selected options (e.g., a check mark) or unavailable options (e.g., dimmed text).

iii) Form Interaction: It is a highly intuitive human-computer interaction method whereby data fields are formatted in a manner similar to paper-based forms. It allows users to fill in the blanks when working with a system. Form interaction is important and effective for both the input and presentation.

iv) Object-Based Interaction: A human-computer interaction method in which symbols are used to represent commands or functions.

The most common method for implementing object-based interaction is through the use of icons. Icons are graphic symbol that represents specific functions within a system. Icons take little space and can be quickly understood by users.

v) Natural Language Interaction: A human-computer interaction method whereby inputs to and outputs from a computer-based application are in a conventional spoken language such as English. It is being applied with both keyboard and voice entry systems.

Designing Interfaces: User Interface is the front end application view to which user interacts in order to use the software. The software becomes more popular if its user interface is attractive, simple to use, responsive, clear to understand and consistent on all interfacing screens.

Effective interface design requires a thorough understanding of each of these concepts.

1) Designing Layouts: For designing layouts standard formats similar to paper-based forms and reports should be used. Screen navigation on data entry screens should be left-to-right, top-to-bottom as on paper forms.

Flexibility and consistency are primary design goals.

2) Structuring data entry:

→ Never require data that are already on-line or that can be computed.

→ Always provide default values when appropriate.

→ Make clear the type of data units requested for entry.

→ Use character replacement when appropriate.

3) Controlling data input:

→ One objective of interface design is to reduce data entry errors.

→ Role of systems analyst is to anticipate user errors and design features into the system's interfaces to avoid, detect and correct data entry mistakes.

4) Providing Feedback:

- Interface design should provide status information as feedback like keeping users informed of what is going on the system.
- It should provide error and warning messages.

5) Providing Help:

- We should try to place ourself in user's place when designing help.
- Help messages should be short and to the point.

Designing Dialogues: The process of designing the overall sequences that users follow to interact with an information system is called dialogue design. A dialogue is the sequence in which information is displayed to and obtained from a user. The dialogue design process consists of three major steps:

1. Designing the dialogue sequence
2. Building a prototype
3. Assessing usability.

A few general rules (guidelines) that should be followed when designing a dialogue are:-

<u>Guideline</u>	<u>Explanation</u>
<u>Consistency:</u>	Dialogues should be consistent in sequence of actions, key strokes, and terminology (e.g. the same labels should be used for the same operations on all screens, and the location of the same information should be the same on all displays).

Shortcuts and Sequence : Allow advanced users to take shortcuts using special keys (e.g., CTRL-C to copy highlighted text). A natural sequence of steps should be followed (e.g., enter first name before last name, if appropriate).

Feedback : Feedback should be provided for every user action (e.g., confirm that a record has been added, rather than simply putting another blank form on the screen).

Closure : Dialogues should be logically grouped and have a beginning, middle, and end (e.g., the last in the sequence of screens should indicate that there are no more screens).

Error Handling : All errors should be detected and reported; suggestions on how to proceed should be made (e.g., suggest why such errors occur and what user can do to correct the error). Synonyms for certain responses should be accepted (e.g., accept either "t", "T" or "TRUE").

Reversal : Dialogues should, when possible, allow the user to reverse actions (e.g., undo a deletion); data should not be deleted without confirmation (e.g., display all the data for a record the user has indicated is to be deleted).

Control : Dialogues should make the user (especially an experienced user) feel in control of the system (e.g., provide a consistent response time at a pace acceptable to the user).

Ease: It should be a simple process for users to enter information and navigate between screens (e.g., provide means to move forward, backward, and to specific screens, such as first and last).

Designing the Dialogue Sequence

First step in dialogue design is to define the sequence. For designing the dialogue sequence we must first gain an understanding of how users might interact with the system. This means we must have a clear understanding of user, task, technological, and environmental characteristics when designing dialogues.

As a designer, once we understand how a user wishes to use a system, we can then transform these activities into a formal dialogue specification.

A formal method for designing and representing dialogues is dialogue diagramming. Dialogue diagrams have only one symbol, a box with three sections;

1. Top: Contains a unique display reference number used by other displays for referencing it.
2. Middle: Contains the name or description of the display.
3. Bottom: Contains display reference numbers that can be accessed from the current display.

Date

	Unique Reference Number of Display	← Top
	Name or Description of Display	← Middle
	Reference Numbers of Return Displays	← Bottom

Building Prototypes and Assessing Usability :

Building dialogue prototypes and assessing usability are often optional activities. Some systems may be very simple and straightforward. Others may be more complex but are extensions to existing systems where dialogue and displays standards have already been established. In either case, we may not be required to build prototypes and do a formal assessment.

Designing Interfaces and Dialogues in Graphical Environments :

Graphical User Interface (GUI) environments have become the de facto for human-computer interaction. When designing graphical interfaces we encounter different issues, so numerous factors must be considered. Some factors are common to all GUI environments, whereas others are specific to the single environment. In most discussions of GUI programming, two rules repeatedly emerge as

Composing the first step to become an effective GUI designer:

- Become an expert of the GUI environment.
- Understand the available resources and how they can

be used.

Common Properties of Windows and Forms In a GUI Environment That can be Active or Inactive

<u>property</u>	<u>Explanation</u>
<u>Modality</u>	: Requires users to resolve the request for information before proceeding (e.g., need to cancel or save before closing a window)
<u>Resizable</u>	: Allows users to resize a window or form (e.g., to make room to see other windows that are also on the screen).
<u>Movable</u>	: Allows users to move a window or form (e.g., to all another window to be seen).
<u>Maximize</u>	: Allows users to expand a window or form to a full-size screen (e.g., to avoid distraction from other active windows or forms).
<u>Minimize</u>	: Allows users to shrink a window or form to an icon (e.g., to get the window out of the way while working on other active windows).
<u>System Menu</u>	: Allows a window or form to also have a system menu to directly access system-level functions (e.g., to save or copy data).